QA Run Report: run_df6051ed92

■ Executive Quality Summary

Overall Quality Score: `35/100`
Current Status: `FAILED`

■■ Core Metrics Breakdown

Reliability Score: `80/100` (Static & Runtime Stability)

Validation Score: `30/100` (Business Logic Coverage)

Hallucination Risk: `0/100` (Ungrounded Logic Detectors)

Resilience Health: `20/100` (Error Handling & Retries)

■ Critical Security & Logic Findings

[MEDIUM] Undefined variable or function reference: Undefined variable or function reference: 'user_id'

[CRITICAL] Application Initialization Failure: The app crashed during import. This usually means top-level dependencies or env vars are missing.

Error: name 'user_id' is not defined

[HIGH] Missing required field: edges: The uploaded automation.json is missing the top-level `edges` field.

[HIGH] Workflow edges are missing: A node/edge workflow was uploaded without `edges`.

[MEDIUM] Workflow node type is not in the approved registry: Node `collector` declares unsupported type `file_collector`.

[MEDIUM] Workflow node type is not in the approved registry: Node `planner` declares unsupported type `llm_planner`.

[MEDIUM] Workflow node type is not in the approved registry: Node `router` declares unsupported type `tool_router`.

[MEDIUM] Workflow node type is not in the approved registry: Node `executor` declares unsupported type `python_runner`.

[MEDIUM] Workflow node type is not in the approved registry: Node `memory` declares unsupported type `vector_memory`.

[MEDIUM] Workflow node type is not in the approved registry: Node `repair` declares unsupported type `self_healer`.

[MEDIUM] Workflow node type is not in the approved registry: Node `approval` declares unsupported type `approval_gate`.

[MEDIUM] Workflow node type is not in the approved registry: Node `notifier` declares unsupported type `email_sender`.

[MEDIUM] Workflow node type is not in the approved registry: Node `finalizer` declares unsupported type `finalizer`.

[HIGH] Missing validation layer: The workflow has no explicit validation node or rule after execution steps.

[MEDIUM] Retry policy is missing: The workflow does not declare retry attempts, delays, or backoff strategy.

■ Autonomous Failure Explainer

Root Cause Analysis

The variable 'user_id' is not defined before being passed to the SQL query, causing a NameError and crashing the application.

User Impact: If this code goes to production, users will experience a complete application failure when trying to access the database, resulting in a poor user experience and potential data loss.

Recommended Fix: Define the 'user_id' variable before using it in the SQL query. This could involve retrieving the user ID from a secure source, such as user input or an authentication system, and assigning it to the 'user_id' variable. Additionally, consider adding input validation and error handling to prevent similar crashes in the future.

■■ Autonomous Repair Strategies

Strategy: Safe Database Connection

Safety Score: 95.0%

Rationale: This works by providing a default database URL in case the environment variable is missing

Suggested Code Patch:

```
import os
import sqlite3
DB_URL = os.getenv("DB_URL", "sqlite:///:memory:")
query = "SELECT * FROM users WHERE id = ?"
try:
    connection = sqlite3.connect(DB_URL)
    cursor = connection.cursor()
    cursor.execute(query, (user_id,))
    # rest of the code...
except sqlite3.Error as e:
    print(f"An error occurred: {e}")
```

Strategy: Type Checking and Validation

Safety Score: 92.0%

Rationale: This works by validating the user_id before executing the query

Suggested Code Patch:

```
import os
import sqlite3
DB_URL = os.getenv("DB_URL", "sqlite:///:memory:")
query = "SELECT * FROM users WHERE id = ?"
if user_id is not None and isinstance(user_id, int):
    try:
        connection = sqlite3.connect(DB_URL)
        cursor = connection.cursor()
        cursor.execute(query, (user_id,))
        # rest of the code...
    except sqlite3.Error as e:
        print(f"An error occurred: {e}")
else:
    print("Invalid user_id")
```

Strategy: Error Handling Enhancement

Safety Score: 90.0%

Rationale: This works by catching specific exceptions and providing more informative error messages

Suggested Code Patch:

```
import os
import sqlite3
DB_URL = os.getenv("DB_URL", "sqlite:///:memory:")
query = "SELECT * FROM users WHERE id = ?"
try:
    connection = sqlite3.connect(DB_URL)
    cursor = connection.cursor()
    cursor.execute(query, (user_id,))
    # rest of the code...
except sqlite3.OperationalError as e:
    print(f"Operational error: {e}")
except sqlite3.ProgrammingError as e:
    print(f"Programming error: {e}")
except Exception as e:
    print(f"An error occurred: {e}")
```

■ Agent Execution Timeline

`13:50:19` **reflection**: Agent reflection active (running)

`13:50:20` **reflection**: Transitioned to reflection (running)

`13:50:22` **failure_explainer**: The variable 'user_id' is not defined before being passed to the SQL query, causing a NameError and crashing the application. (success)

`13:50:22` **memory_retriever**: Agent memory_retriever active (running)

`13:50:22` **memory_retriever**: Transitioned to memory_retriever (running)

`13:50:23` **memory_retriever**: Retrieved 3 past fixes. (success)

`13:50:23` **self_heal_router**: Agent self_heal_router active (running)

`13:50:23` **self_heal_router**: Transitioned to self_heal_router (running)

`13:50:25` **self_heal_router**: Proposed 3 fixes (success)

`13:50:26` **memory_writer**: Agent memory_writer active (running)

`13:50:26` **memory_writer**: Transitioned to memory_writer (running)

`13:50:30` **retry_or_replan**: Agent retry_or_replan active (running)

`13:50:30` **retry_or_replan**: Transitioned to retry_or_replan (running)

`13:50:30` **finalizer**: Agent finalizer active (running)

`13:50:30` **finalizer**: Transitioned to finalizer (running)